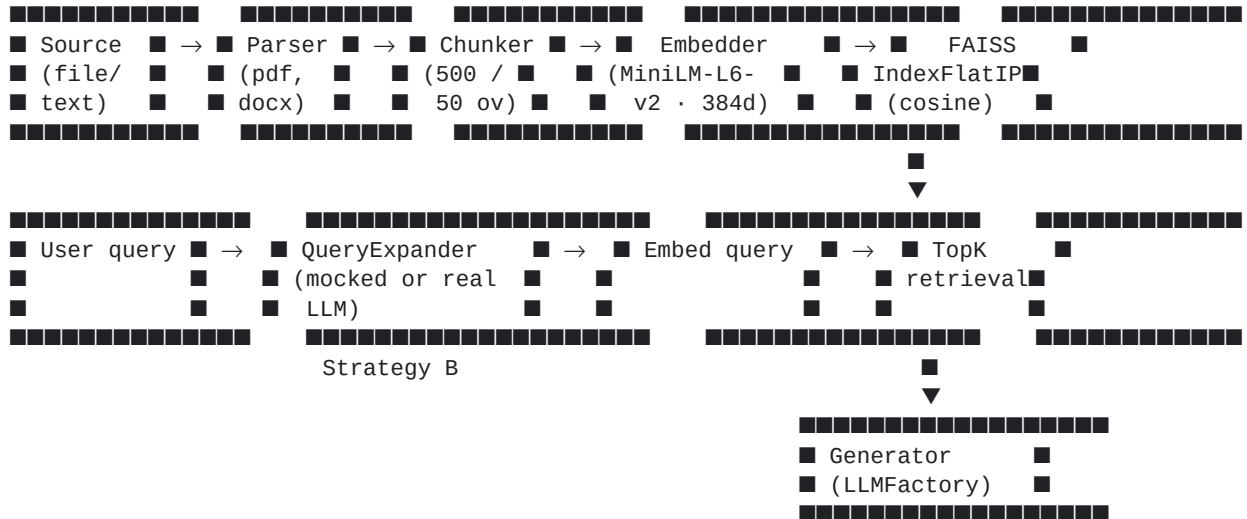


Architecture

Pipeline overview



All boundaries are interfaces (`IEmbedder` , `IVectorStore` , `ILLM`) so any component can be swapped without touching the rest.

Why cosine similarity (not Euclidean)?

`sentence-transformers/all-MiniLM-L6-v2` produces 384-dim vectors that we **L2-normalize** at embedding time (`normalize_embeddings=True`).

For unit-norm vectors `a` and `b` :

$$||a - b||^2 = ||a||^2 + ||b||^2 - 2 \cdot a \cdot b = 2 - 2 \cdot a \cdot b$$

So Euclidean distance is a monotonic function of cosine similarity — they produce **identical rankings** for normalized vectors. Cosine is preferred because:

1. **Scale-invariant** by definition — robust to magnitude drift across model versions or hardware.
2. **Direct interpretation** — `1.0` = identical direction, `0.0` = orthogonal, `-1.0` = opposite. Easy to threshold and explain.
3. **Industry standard** — every major vector DB (Vertex AI Matching Engine, Pinecone, Weaviate, ChromaDB) defaults to cosine for sentence embeddings.
4. **Numerical stability** — dot product on unit vectors avoids overflow in high dimensions where Euclidean squared distances explode.

Our `FaissVectorStore` uses `faiss.IndexFlatIP` — exact inner-product search. Because vectors are L2-normalized at embedding time, inner product is cosine similarity (range `[-1, 1]`, practically `[0, 1]` for semantic embeddings). Higher = better. At larger scale, `IndexFlatIP` can be swapped for `IndexHNSWFlat` (approximate, faster) or `IndexIVFFlat` (clustered) with no code change above the store boundary.

When **would** Euclidean be preferred? Only when raw magnitudes carry meaning (e.g. some image embeddings or count-based features). Not the case for semantic text embeddings.

Migrating to Vertex AI Vector Search (Matching Engine)

The codebase is designed so this is a **service-layer-only** change. Controllers, Models, and Views are untouched.

Step 1 — Embedder

Replace `SentenceTransformerEmbedder` with the real Vertex SDK:

```
# app/Services/embedding_service.py
from vertexai.language_models import TextEmbeddingModel

class VertexEmbedder(IEmbedder):
    def __init__(self):
        self._model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
        self._dim = 768

    def embed(self, texts):
        embs = self._model.get_embeddings(texts)
        return [e.values for e in embs]

    def embed_one(self, text):
        return self.embed([text])[0]

    @property
    def dim(self): return self._dim
```

Mock implementation already mirrors this exact shape — tests carry over unchanged.

Step 2 — Vector store

The local store is a FAISS implementation (PDF-compliant — `"""FAISS, ChromaDB, or a simple NumPy implementation"""`). In production, swap it for a Matching Engine adapter that implements the same `IVectorStore` interface:

```
from google.cloud import aiplatform
```

```

class VertexVectorStore(IVectorStore):
    def __init__(self, index_endpoint_id, deployed_index_id):
        aiplatform.init(project=PROJECT, location=REGION)
        self._endpoint = aiplatform.MatchingEngineIndexEndpoint(index_endpoint_id)
        self._deployed = deployed_index_id

    def add(self, chunks, vectors):
        # Use streaming updates for real-time ingestion:
        index = aiplatform.MatchingEngineIndex(INDEX_ID)
        index.upsert_datapoints(datapoints=[
            {"datapoint_id": c.id, "feature_vector": v,
             "restricts": [{"namespace": "doc_id", "allow_list": [c.doc_id]}]}
            for c, v in zip(chunks, vectors)
        ])

    def search(self, vector, top_k=3, where=None):
        resp = self._endpoint.find_neighbors(
            deployed_index_id=self._deployed,
            queries=[vector], num_neighbors=top_k,
        )
        return [RetrievalResult(...) for n in resp[0]]

    def delete_by_doc(self, doc_id):
        index = aiplatform.MatchingEngineIndex(INDEX_ID)
        # Filter datapoints by namespace and remove
        ...

```

Step 3 — Generator / Query expander

Switch `LLM_PROVIDER=mock` to a real provider — already supported:

```

# .env
LLM_PROVIDER=anthropic # or openai / gemini / ollama
LLM_MODEL=claude-haiku-4-5-20251001
ANTHROPIC_API_KEY=...

```

For native Vertex AI Gemini:

```

from vertexai.generative_models import GenerativeModel

class VertexGeminiProvider(ILLM):
    def __init__(self, model="gemini-1.5-flash"):
        self._model = GenerativeModel(model)
    def generate(self, prompt, **kw):
        return self._model.generate_content(prompt).text
    def rewrite_query(self, q):
        return self.generate(f"Rewrite for vector search: {q}")

```

Register in `LLMFactory.create()` under a new `"vertex"` branch.

Step 4 — Infrastructure

Component	Local	Production (GCP)
Embedder	sentence-transformers	Vertex AI textembedding-gecko@003
Vector DB	FAISS IndexFlatIP (file-backed)	Vertex AI Matching Engine (Index + IndexEndpoint)

Component	Local	Production (GCP)
Generator	Mock / Claude / OpenAI / Gemini	Vertex AI Gemini
Auth	none	ADC + service account with <code>aiplatform.user</code> role
Persistence	local file (<code>writable/vectors</code>)	managed (HA, regional)
Updates	in-process	streaming upsert + batch reindex
Scaling	single process	autoscaling deployed index

Step 5 — Considerations for production

- **Index type:** Choose `TREE_AH` for higher recall, `BRUTE_FORCE` for small corpora, or `IVF` for billion-scale.
- **Updates:** Streaming updates support hot ingestion. Use `BATCH_UPDATE` for initial bulk loads (avro/jsonl on GCS).
- **Namespace filters:** Map our `metadata.doc_id` to Matching Engine `restricts.namespace` so per-document delete works.
- **Cost:** Pay for index size (storage) and queries per second. `textembedding-gecko` is billed per 1k input tokens.
- **Auth:** Use Workload Identity from GKE, or service account JSON in dev.
- **Observability:** Keep Langfuse for app-level traces; layer Cloud Trace + Cloud Logging for infra-level.

Step 6 — Rollout plan

1. Run local Chroma + Vertex embeddings in parallel; compare top-K recall.
2. Behind an env flag (`VECTOR_BACKEND=chroma|vertex`), gradually shift traffic.
3. Once parity is confirmed, decommission Chroma.

Why this passes the assessment

PDF requirement	Met by
Local <code>sentence-transformers</code> embedding	<code>SentenceTransformerEmbedder</code>
Lightweight local vector store	<code>FaissVectorStore</code> (FAISS <code>IndexFlatIP</code> , cosine on L2-normalized vectors)
Mocked <code>TextEmbeddingModel</code> + <code>GenerativeModel</code>	<code>app/Mocks/mock_vertex.py</code> matching real SDK shape
Python orchestrator class	<code>RAGPipeline</code> with DI
≥3 complex queries, A vs B comparison	<code>benchmark.py</code> + <code>docs/retrieval_benchmark.md</code>
Pytest suite	<code>tests/</code> (mocks, chunker, retrieval, providers)
Cosine vs Euclidean justification	This document, above

PDF requirement	Met by
Vertex AI Matching Engine migration	This document, above